

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación

Laboratorio 3.2

Sistema de Parquadero – Configuración de Server-Sent Events (SSE) en ms-tickets

Autor: Cáceres Villagómez Germán Patricio

NRC: 30731 – Aplicaciones Distribuidas

Docente: Ing. Ángel Cudco, Mgs.

Fecha: 19 de julio de 2026

Sangolquí – Ecuador

Índice

Resumen	2
1. Introducción	3
2. Objetivos	3
Objetivo General	3
Objetivos Específicos	4
3. Desarrollo	5
3.1. Fundamento: Server-Sent Events	5
3.2. Módulo SSE en ms-tickets	5
3.3. Integración con el ciclo de vida del ticket	6
3.4. Autenticación de servicio a servicio	7
3.5. Configuración del API Gateway para tráfico SSE	7
3.6. Consumo en el cliente (dashboard)	8
3.7. Verificación funcional	9
4. Conclusiones	11
5. Recomendaciones	12
Bibliografía	13

Resumen

El presente informe documenta la configuración e implementación de un canal de comunicación en tiempo real basado en *Server-Sent Events* (SSE) dentro del microservicio `ms-tickets`, componente del sistema distribuido *Parqueadero Inteligente*. Este mecanismo permite que el *dashboard* del cliente refleje, sin necesidad de recargar la página, los cambios de estado de los espacios de estacionamiento (disponible, ocupado, reservado, en mantenimiento o inactivo) en el instante en que ocurren, como consecuencia del ciclo de vida de un ticket de estacionamiento. Se describe el patrón interno de publicación-suscripción implementado con RxJS, la integración entre `ms-tickets` y el microservicio `zonas` para sincronizar el estado real de cada espacio, la solución adoptada para autenticar dicha comunicación interna una vez que se incorporó seguridad basada en roles al sistema, y la configuración particular que debió aplicarse en el API Gateway (Kong) para permitir el paso de un flujo SSE sin que su comportamiento de *buffering* por defecto interrumpiera la transmisión. El informe cierra con la verificación funcional del flujo completo: creación y cierre de un ticket, actualización del espacio correspondiente, emisión del evento y recepción en el cliente conectado.

1 Introducción

En un sistema de gestión de estacionamientos, la disponibilidad de los espacios es un dato que cambia con alta frecuencia y cuya vigencia es crítica: un panel de monitoreo que muestre información desactualizada puede llevar a un usuario o a un operador a asumir que un espacio está libre cuando en realidad ya fue ocupado segundos antes. La forma tradicional de mantener actualizada una interfaz web ante estos cambios es el *polling*, es decir, que el cliente consulte periódicamente al servidor si hay novedades; sin embargo, esta técnica introduce una latencia igual al intervalo de consulta y genera tráfico innecesario cuando no hay cambios que reportar.

Server-Sent Events (SSE) es un estándar del World Wide Web Consortium que permite a un servidor mantener abierta una única conexión HTTP y enviar, a través de ella, actualizaciones al cliente en el momento en que se producen, sin que este deba solicitarlas repetidamente. A diferencia de WebSockets, que habilita comunicación bidireccional, SSE está diseñado específicamente para el caso – mucho más común en aplicaciones de monitoreo– en que solo el servidor necesita empujar información hacia el cliente.

En el sistema *Parqueadero Inteligente*, el microservicio **ms-tickets** es responsable de gestionar el ciclo de vida de los tickets de estacionamiento (ingreso, actualización, salida y anulación), y cada una de estas operaciones provoca, como efecto colateral, un cambio en el estado del espacio asociado dentro del microservicio **zonas** (por ejemplo, de **DISPONIBLE** a **OCUPADO** al crear un ticket). Este informe documenta cómo dicho cambio de estado se propaga en tiempo real hacia el *dashboard* web mediante SSE, y los ajustes de seguridad y de infraestructura que fue necesario realizar para que ese flujo funcionara de manera confiable una vez que se reforzaron los controles de acceso del sistema.

2 Objetivos

Objetivo General

Implementar y documentar la configuración de comunicación en tiempo real mediante *Server-Sent Events* en el microservicio **ms-tickets**, de manera que los cambios de estado de los espacios de estacionamiento se reflejen instantáneamente en el *dashboard* del sistema *Parqueadero Inteligente*.

Objetivos Específicos

1. Describir el patrón de publicación-suscripción interno, basado en un **Subject** de RxJS, que emplea **ms-tickets** para emitir eventos SSE hacia los clientes conectados al endpoint `/sse/espacios`.
2. Configurar la integración entre el ciclo de vida de un ticket (creación y cierre) y la actualización del estado del espacio correspondiente en el microservicio **zonas**, incorporando la autenticación de servicio a servicio requerida tras la incorporación de seguridad basada en roles a dicho microservicio.
3. Adaptar la configuración del *API Gateway* Kong para permitir el paso de un flujo SSE sin que el comportamiento de *buffering* por defecto lo interrumpa, y verificar la recepción en tiempo real de los eventos emitidos en el *dashboard* cliente.

3 Desarrollo

3.1 Fundamento: Server-Sent Events

SSE es un mecanismo definido dentro del estándar HTML por el WHATWG que permite a un servidor enviar eventos de texto a un cliente a través de una única conexión HTTP persistente, usando el tipo de contenido `text/event-stream`. Del lado del navegador, la interfaz `EventSource` de JavaScript se encarga de abrir la conexión, interpretar cada bloque de datos recibido y reintentar automáticamente la conexión en caso de corte, sin que la aplicación deba implementar dicha lógica manualmente. A diferencia de WebSockets, SSE utiliza HTTP estándar (lo que simplifica su paso a través de *proxies* y balanceadores) y es unidireccional: únicamente el servidor envía datos, característica suficiente para el caso de uso de este sistema, donde el cliente solo necesita *recibir* actualizaciones del estado de los espacios.

3.2 Módulo SSE en ms-tickets

Dentro de `ms-tickets` se implementó un módulo dedicado (`SseModule`) compuesto por un servicio y un controlador. El servicio (`SseService`) encapsula un `Subject` de RxJS, que actúa como el canal interno de publicación-suscripción: cualquier parte de la aplicación puede invocar `emitEvent(tipo, datos)` para publicar un evento, y el controlador se suscribe a ese mismo flujo para reenviarlo a los clientes HTTP conectados.

```
1 export interface SseEvent {
2   type: string;
3   data: any;
4 }
5
6 @Injectable()
7 export class SseService {
8   private eventSubject = new Subject<SseEvent>();
9
10  emitEvent(type: string, data: any) {
11    this.eventSubject.next({ type, data });
12  }
13
14  getEventStream() {
15    return this.eventSubject.asObservable();
16  }
17 }
```

Listing 1: Servicio SSE basado en un Subject de RxJS (sse.service.ts)

El controlador expone la ruta `GET /sse/espacios` mediante el decorador `@Sse` de NestJS, el cual configura automáticamente los encabezados HTTP necesarios (`Content-Type: text/event-stream`) y mantiene la conexión abierta mientras transforma cada valor emitido por el `Observable` en un cuadro de evento SSE:

```
1 @Controller('sse')
2 export class SseController {
3   constructor(private readonly sseService: SseService) {}
4
5   @Sse('espacios')
6   streamEspacios(): Observable<MessageEvent> {
7     return this.sseService.getEventStream().pipe(
8       map((event) => ({
9         data: JSON.stringify(event),
10        type: event.type,
11      })),
12     );
13   }
14 }
```

Listing 2: Controlador que expone el flujo SSE (sse.controller.ts)

3.3 Integración con el ciclo de vida del ticket

El punto donde se origina cada evento SSE es el servicio `ZoneIntegrationService`, responsable de mantener sincronizado el estado del espacio en `zonas` con las operaciones que ocurren sobre un ticket. Al crear un ticket, dicho servicio marca el espacio como `OCUPADO`; al registrarse la salida, la anulación o la eliminación de un ticket activo, lo marca nuevamente como `DISPONIBLE`. Inmediatamente después de confirmar la actualización en `zonas`, el mismo método invoca `SseService.emitEvent`, cerrando el ciclo entre la acción de negocio y la notificación en tiempo real:

```
1 await firstValueFrom(
2   this.httpService.put(
3     `${this.zoneServiceUrl}/api/v1/espacios/${idEspacio}`,
4     { idZona: espacio.idZona, descripcion: espacio.descripcion,
5       tipo: espacio.tipo, estado },
6     { headers: { Authorization: `Bearer ${this.serviceToken()}` } },
7   ),
8 );
9
10 this.sseService.emitEvent('Espacio actualizado', { id: idEspacio, estado });
```

Listing 3: Emisión del evento SSE tras sincronizar el estado del espacio (zone-integration.service.ts)

3.4 Autenticación de servicio a servicio

El microservicio **zonas** incorpora, como parte del endurecimiento de seguridad del sistema, un filtro que exige un token JWT válido con rol **admin**, **root** o **recaudador** para cualquier operación de escritura sobre zonas y espacios. Dado que la llamada que **ms-tickets** realiza hacia **zonas** para actualizar el estado de un espacio no proviene de un usuario final sino de una comunicación interna entre servicios, dicha llamada dejó de funcionar al aplicarse el filtro, pues nunca incluía un encabezado de autorización.

La solución adoptada consiste en que **ms-tickets** firme su propio token, empleando el mismo secreto compartido (**JWT_SECRET**) que utiliza el microservicio **personas** para emitir los tokens de los usuarios reales, y lo declare bajo una identidad de servicio explícita:

```
1 private serviceToken(): string {  
2     return this.jwtService.sign({  
3         sub: 'ms-tickets-service',  
4         username: 'ms-tickets-service',  
5         roles: ['admin'],  
6     });  
7 }
```

Listing 4: Emisión de un token de identidad de servicio (zone-integration.service.ts)

Gracias a esta identidad explícita, cada actualización de espacio originada por **ms-tickets** queda registrada en la auditoría del sistema bajo el usuario **ms-tickets-service**, lo que permite distinguirla claramente de una acción realizada por un administrador humano.

3.5 Configuración del API Gateway para tráfico SSE

Todo el tráfico hacia los microservicios del sistema, incluyendo el flujo SSE, se enruta a través del *API Gateway* Kong. Por defecto, Kong – como la mayoría de *proxies* inversos– almacena en búfer la respuesta completa del servicio de origen antes de reenviarla al cliente, comportamiento pensado para respuestas HTTP convencionales de tamaño finito. Un flujo SSE, sin embargo, nunca "termina": permanece abierto indefinidamente emitiendo fragmentos de datos según van ocurriendo los eventos, por lo que dicho *buffering* impide que la información llegue al cliente en tiempo real.

Para resolverlo, la ruta registrada en Kong para el endpoint **/sse** deshabilita explícitamente el almacenamiento en búfer tanto de la petición como de la respuesta:

```
1 curl -s -X POST "$KONG_ADMIN/services/tickets-service/routes" \  
2     -d '{  
3         "name": "tickets-sse",
```



```
4  "paths": ["/sse"],
5  "strip_path": false,
6  "request_buffering": false,
7  "response_buffering": false
8  },'
```

Listing 5: Registro de la ruta SSE en Kong sin buffering (setup-kong.sh)

3.6 Consumo en el cliente (dashboard)

El *dashboard* web, servido de forma independiente como un contenedor Nginx dentro de su propio proyecto Docker Compose, se conecta al flujo SSE mediante la interfaz nativa `EventSource` del navegador, apuntando siempre hacia el *gateway* Kong:

```
1  const conectarSSE = () => {
2    const eventSource = new EventSource(SSE_URL);
3
4    eventSource.onopen = () => setConnectionStatus(true);
5
6    eventSource.onmessage = (event) => {
7      JSON.parse(event.data);
8      cargarEspacios(); // recarga el listado completo de espacios
9    };
10
11   eventSource.onerror = () => {
12     setConnectionStatus(false);
13     eventSource.close();
14     setTimeout(conectarSSE, 5000); // reintento tras 5 segundos
15   };
16 };
```

Listing 6: Conexión al flujo SSE desde el dashboard (app.js)

Ante cualquier evento recibido, el cliente vuelve a consultar el listado completo de espacios en lugar de aplicar únicamente el cambio puntual reportado; esta estrategia, aunque más simple que una actualización incremental, tiene la ventaja de reflejar también la creación de nuevos espacios y de mantener la interfaz consistente ante eventos fuera de orden. Como salvaguarda adicional, el *dashboard* también refresca la información cada 30 segundos mediante un *polling* de respaldo, de modo que una eventual pérdida silenciosa de la conexión SSE no deje la interfaz permanentemente desactualizada. Cada espacio se colorea según su estado (DISPONIBLE en verde, OCUPADO en rojo, RESERVADO en violeta, MANTENIMIENTO en ámbar, INACTIVO en gris), reforzando visualmente la información recibida en tiempo real.

La Figura 1 corresponde a una captura real del *dashboard* en ejecución, tomada después de haber actualizado, mediante peticiones directas al microservicio *zonas*, el estado de distintos

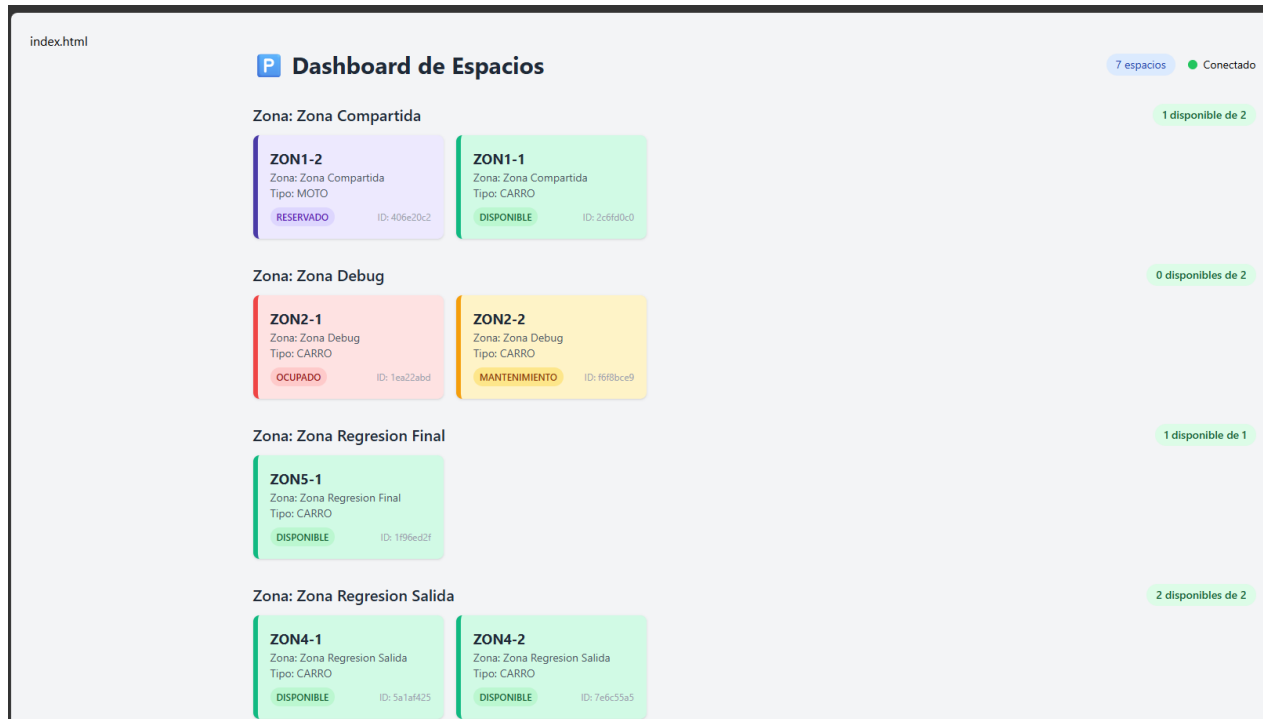


Figura 1: Dashboard de espacios (`localhost:8085`) mostrando simultáneamente los cuatro estados con color distintivo – OCUPADO en rojo, RESERVADO en violeta, MANTENIMIENTO en ámbar y DISPONIBLE en verde– junto con el indicador de conexión SSE activa (*Conectado*) en la esquina superior derecha.

espacios de prueba. El indicador *Conectado* en la parte superior confirma que la conexión `EventSource` hacia `/sse/espacios` se mantenía activa en el momento de la captura, y la diferenciación cromática entre las cuatro tarjetas permite verificar, de un solo vistazo, que cada valor del enumerado `EstadoEspacio` recibe un tratamiento visual propio y no se confunde con los demás.

3.7 Verificación funcional

Se verificó el flujo completo de extremo a extremo:

- Con una conexión abierta al endpoint `GET /sse/espacios` a través de Kong, se confirmó mediante los encabezados de respuesta (`Content-Type: text/event-stream`, sin indicios de buffering) que el flujo permanecía activo indefinidamente.
- Al registrar la creación de un ticket para un espacio disponible, se observó en la conexión SSE la llegada inmediata de un evento `Espacio actualizado` con el nuevo estado

OCUPADO, y se confirmó mediante una consulta directa al microservicio `zonas` que el espacio reflejaba efectivamente dicho estado.

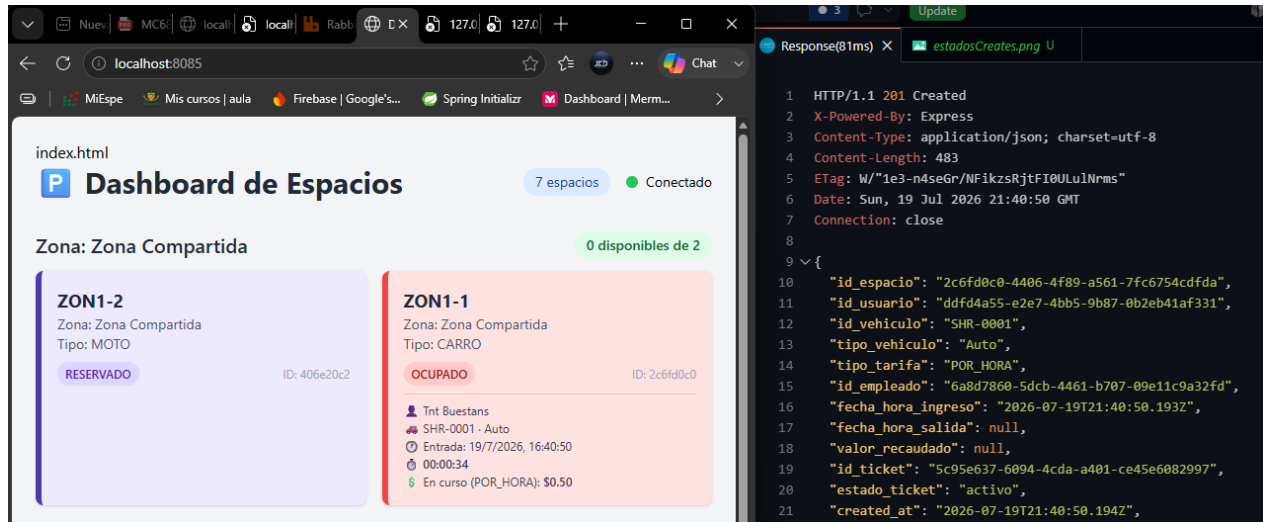


Figura 2: Respuesta 201 **Created** de `POST /tickets` (panel derecho) y actualización inmediata del *dashboard* (panel izquierdo): el espacio `ZON1-1` pasa a **OCUPADO** y su tarjeta muestra, en tiempo real, el usuario, la placa, el tipo de vehículo, la hora de ingreso, el contador de tiempo transcurrido y el valor en curso según la tarifa `POR_HORA` elegida al crear el ticket.

La Figura 2 evidencia que, apenas confirmada la creación del ticket, el evento SSE emitido por `ZoneIntegrationService` ya se reflejó en el *dashboard* sin necesidad de recargar la página manualmente ni esperar al *polling* de respaldo.

- Al registrar la salida del vehículo asociado a ese ticket, se observó un segundo evento SSE indicando el regreso del espacio al estado **DISPONIBLE**, cerrando el ciclo completo.

La Figura 3 corresponde al mismo ticket ilustrado en la Figura 2 una vez registrada su salida: el cuerpo de la respuesta confirma que `valor_recaudado` se calculó del lado del servidor a partir del plan de tarifa y el tiempo real transcurrido, sin intervención manual, y el *dashboard* liberó el espacio en el mismo instante gracias al evento SSE correspondiente.

- Se verificó en el registro de auditoría (`ms-audit`) que ambas actualizaciones de espacio quedaron atribuidas al usuario de servicio `ms-tickets-service`, confirmando el correcto funcionamiento de la autenticación de servicio a servicio descrita.

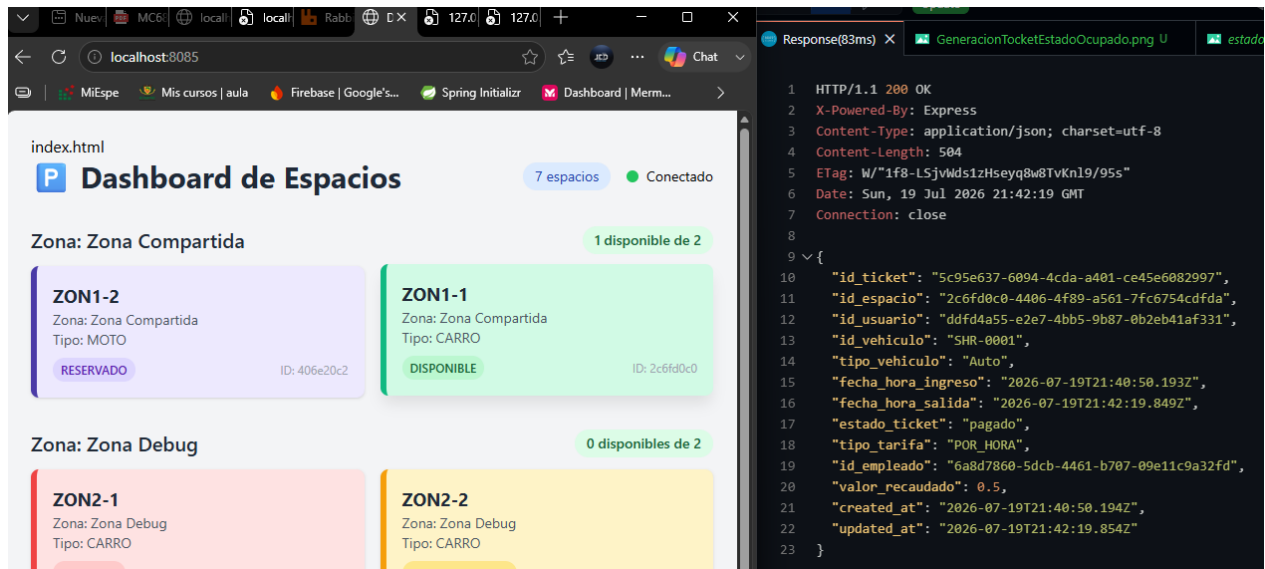


Figura 3: Respuesta 200 OK de PATCH /tickets/:id/salida (panel derecho), con estado_ticket en pagado y valor_recaudado calculado automáticamente por el servidor, junto con el dashboard (panel izquierdo) reflejando de inmediato el regreso del espacio ZON1-1 al estado DISPONIBLE.

4 Conclusiones

- Server-Sent Events constituye una solución adecuada y de bajo costo de implementación para el caso de actualizaciones unidireccionales de baja a media frecuencia, como la notificación de cambios de estado de un espacio de estacionamiento, evitando la sobrecarga que introduciría un esquema de *polling* constante.
- El uso de un Subject de RxJS como canal interno de publicación-suscripción simplifica notablemente la propagación de eventos entre distintos servicios de una misma aplicación NestJS, desacoplando el punto donde ocurre el cambio de negocio del punto donde este se transmite al cliente.
- El endurecimiento de la seguridad de un microservicio mediante controles de autenticación y autorización puede romper, de forma no evidente, integraciones internas preexistentes entre servicios si estas no se identifican y autentican de manera explícita; en este caso, resolverlo mediante una identidad de servicio firmada con el secreto compartido permitió mantener tanto la seguridad como la trazabilidad de dichas llamadas.
- El comportamiento por defecto de un *API Gateway* – en particular el almacenamiento en búfer de las respuestas– no es neutro frente a todos los tipos de tráfico, y debe

revisarse y ajustarse explícitamente cuando se introduce un patrón de comunicación distinto al de petición-respuesta convencional, como es el caso de un flujo SSE.

- Combinar un canal de actualización en tiempo real con un mecanismo de respaldo basado en *polling* periódico aporta resiliencia a la interfaz de usuario frente a fallos silenciosos de la conexión primaria.

5 Recomendaciones

- Sustituir, en un entorno de producción, el token de servicio estático firmado con el secreto compartido por credenciales de vida corta (por ejemplo, tokens de servicio con expiración breve y renovación automática) o por autenticación mutua TLS entre microservicios, reduciendo el impacto de una eventual filtración del secreto.
- Evaluar la migración hacia una actualización incremental del estado en el cliente (aplicar únicamente el cambio reportado por el evento) en lugar de recargar el listado completo de espacios, especialmente si el número de espacios gestionados por el sistema creciera de forma significativa.
- Incorporar un identificador de correlación en cada evento SSE que permita rastrear, en los registros de auditoría, la operación de negocio exacta que originó una actualización de espacio determinada.
- Documentar explícitamente, dentro del código y de la configuración de despliegue, las identidades de servicio utilizadas para comunicación interna (como `ms-tickets-service`), de manera que sean fácilmente reconocibles y auditables por el equipo de operaciones.
- Considerar la adopción de WebSockets únicamente si en el futuro se requiriera que el *dashboard* envíe información hacia el servidor en el mismo canal; mientras el requerimiento permanezca unidireccional, SSE sigue siendo la alternativa más simple y adecuada.

Referencias

- [1] WHATWG. (2024). *HTML Living Standard – Server-sent events*. <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [2] MDN Web Docs. (2024). *Using server-sent events*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
- [3] NestJS. (2024). *Server-Sent Events*. Documentación oficial. <https://docs.nestjs.com/techniques/server-sent-events>
- [4] Kong Inc. (2024). *Kong Gateway Documentation – Proxy Reference*. <https://docs.konghq.com>
- [5] Jones, M., Bradley, J., y Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc7519>